ESP32-Based Healthcare Monitoring System with ECG Analysis, Edge AI, and Fall Detection

Abstract

This project presents a comprehensive ESP32-based healthcare monitoring system designed for continuous real-time monitoring of critical health parameters and detection of medical emergencies. The system integrates multiple biomedical sensors including an ECG sensor for cardiac rhythm analysis, a MAX30100 sensor for blood oxygen saturation (SpO2) measurement, an NTC thermistor for body temperature monitoring, and an MPU6050 accelerometer for fall detection. The ESP32 microcontroller serves as the central processing unit, collecting data from all sensors, performing edge AI inference, and wirelessly transmitting results to a mobile application via its built-in Bluetooth. ECG signals are processed to extract RR intervals (the time between successive R-peaks), which are then fed into a logistic regression machine learning model trained in Google Colab. The model classifies the cardiac condition into three categories: Normal, Tachycardia (abnormally high heart rate), or Bradycardia (abnormally low heart rate). The MAX30100 sensor is used exclusively for SpO2 measurement, the NTC thermistor provides body temperature readings, and the MPU6050 detects falls through abrupt acceleration patterns. All sensor data and ML predictions are displayed on a mobile app via Serial Bluetooth Terminal, enabling continuous remote monitoring. The entire system is powered by a 2-cell Li-ion battery with a buck converter providing stable 5V for the ESP32 and sensors. This integrated solution addresses the growing need for affordable, portable, and intelligent healthcare monitoring devices suitable for home use, elderly care, and telemedicine applications.

Introduction

Cardiovascular diseases (CVDs) remain the leading cause of death globally, with an estimated 17.9 million lives lost each year, according to the World Health Organization. Early detection of cardiac abnormalities such as tachycardia (heart rate > 100 BPM) and bradycardia (heart rate < 60 BPM) is critical for timely medical intervention and prevention of severe complications. Similarly, falls among the elderly represent a major health risk, with approximately 28-35% of people aged 65 and above experiencing at least one fall annually, often leading to fractures, hospitalization, and reduced quality of life. Traditional health monitoring requires periodic visits to clinics and hospitals, limiting continuous observation and delaying detection of transient events. The increasing availability of low-cost microcontrollers, biomedical sensors, and machine learning frameworks has created opportunities for developing affordable, wearable healthcare monitoring systems that provide continuous, real-time health assessment outside clinical settings.

This project addresses these healthcare challenges by developing a comprehensive ESP32-based monitoring system that combines cardiac rhythm analysis, blood oxygen monitoring, body temperature measurement, and fall detection into a single, portable device. The ESP32 is an ideal platform for this application, featuring a dual-core processor, built-in Bluetooth and WiFi, multiple ADC channels, and low power consumption. Its processing capabilities make it suitable for running edge AI inference without requiring cloud connectivity, ensuring privacy and low-latency response.

The ECG sensor (such as AD8232) is connected to the ESP32's analog input and continuously acquires cardiac electrical signals. The raw ECG signal is processed using digital filtering and peak detection algorithms to identify R-peaks, from which RR intervals are calculated. These RR intervals serve as the primary feature for cardiac classification. A logistic regression model, trained in Google Colab using a dataset of RR intervals labeled with corresponding cardiac conditions, is embedded into the ESP32 firmware. The model uses one-vs-rest classification with three classes: Normal, Tachycardia, and Bradycardia. The trained weights, intercepts, and normalization parameters (mean, standard deviation) are hardcoded as constants in the firmware, enabling real-time inference on each detected heartbeat.

The MAX30100 sensor is a pulse oximeter and heart rate sensor that measures SpO2 through optical sensing (red and infrared LEDs). In this project, only the SpO2 value is used, providing additional insight into respiratory and cardiovascular health. The NTC thermistor, connected in a voltage divider with a fixed resistor, measures body temperature via the Steinhart-Hart equation. Temperature is read from the analog pin and converted to Celsius. The MPU6050, a 6-axis accelerometer and gyroscope, is used for fall detection. The system monitors the acceleration vector magnitude and detects sudden, abrupt changes characteristic of falls (e.g., free-fall followed by high-impact acceleration). Upon detecting a fall, the system immediately alerts the user and can optionally send an emergency notification.

All sensor data—ECG-derived heart rate, RR interval, cardiac classification, SpO2, body temperature, and fall status—are transmitted to a mobile app via the ESP32's built-in

Bluetooth using the Serial Bluetooth Terminal app. This enables continuous monitoring by the user, family members, or caregivers. The 2-cell Li-ion battery (7.4V nominal) with a buck converter provides stable 5V for the ESP32 and sensors, ensuring portable, extended operation. This project demonstrates how affordable embedded systems and edge AI can be combined to create a comprehensive, intelligent healthcare monitoring solution suitable for home, elderly care, and telemedicine applications.

Problem Statement

- Cardiovascular diseases are the leading cause of death globally, requiring early detection of arrhythmias
- Tachycardia and bradycardia are common cardiac abnormalities that can be detected from heart rate patterns
- Traditional ECG monitoring requires clinical visits and expensive equipment, limiting continuous monitoring
- Falls among the elderly cause serious injuries and reduced quality of life, often going unreported
- Blood oxygen (SpO2) and body temperature are vital signs that provide important health insights
- Existing healthcare monitoring devices are expensive, bulky, or require cloud connectivity
- Edge AI can provide real-time, privacy-preserving health assessments without internet dependency
- There is a gap between advanced machine learning and affordable healthcare devices
- Continuous, portable monitoring can enable early intervention and improve health outcomes
- Affordable, open-source healthcare solutions can democratize access to health monitoring

Objectives

- Design and implement an ESP32-based healthcare monitoring system with multiple biomedical sensors
- Acquire ECG signals and extract RR intervals using digital signal processing
- Train a logistic regression model in Google Colab for cardiac classification (Normal, Tachycardia, Bradycardia)
- Deploy the trained model on ESP32 for real-time edge AI inference
- Integrate MAX30100 for SpO2 measurement (only SpO2, not heart rate)
- Integrate NTC thermistor for body temperature measurement using Steinhart-Hart equation
- Integrate MPU6050 for fall detection using abrupt acceleration detection
- Transmit all sensor data and ML predictions to a mobile app via ESP32 built-in Bluetooth
- Power the system with 2-cell Li-ion battery and buck converter for 5V
- Provide a cost-effective, portable, and intelligent healthcare monitoring solution

Methodology

System Architecture

The healthcare monitoring system is built around the ESP32 microcontroller, which coordinates data acquisition from all sensors, performs edge AI inference, and transmits results via Bluetooth. The ECG sensor connects to an analog pin, the MAX30100 and MPU6050 connect via I2C, and the NTC thermistor connects to another analog pin. The 2-cell Li-ion battery with buck converter provides stable 5V. The system continuously monitors all parameters and sends formatted data to the mobile app.

Logistic Regression Model Training (Google Colab)

- Dataset: RR intervals labeled as Normal, Tachycardia, Bradycardia (synthetic or clinical)
- Features: RR interval (ms) normalized using training mean and standard deviation
- Model: Logistic Regression with one-vs-rest for 3 classes
- Output: Class probabilities for each condition
- Extracted Parameters: Means, stds, weights (3×1), intercepts (3×1)

:

Advantages

- Comprehensive Health Monitoring: Simultaneous monitoring of cardiac rhythm, SpO2, temperature, and falls
- Edge AI: Logistic regression on ESP32 provides real-time, cloud-independent classification
- Bluetooth Connectivity: Easy data transmission to mobile app without internet
- Portable: Battery-powered with 2-cell Li-ion and buck converter for extended operation
- Early Detection: Identifies tachycardia and bradycardia before critical events
- Fall Detection: Automatic detection of falls for immediate assistance
- Cost-effective: Affordable components compared to commercial health monitors
- Privacy-Preserving: No cloud dependency; data stays local
- Open Source: Customizable and extensible
- User-friendly: Simple mobile app interface for continuous monitoring

Disadvantages

- ECG Signal Quality: Subject to motion artifacts and noise; requires proper electrode placement
- Single-Feature ML Model: Logistic regression uses only RR interval, limiting accuracy
- SpO2 Only from MAX30100: Heart rate not used from this sensor (redundancy avoided)
- No Real-Time Waveform Display: Mobile app shows numeric values, not ECG waveform
- Battery Life: 2-cell Li-ion limits continuous operation to several hours
- No Emergency Alert System: No SMS or GSM for remote alerts (Bluetooth only)
- Fall Detection False Positives: Abrupt movements may trigger false alarms
- Limited Clinical Validation: Not a medical-grade device; for informational purposes
- Bluetooth Range: Limited to ~10 meters from mobile device
- No Data Storage: No SD card or cloud logging for historical review

Hardware Requirements

- ESP32 Development Board
- ECG Sensor Module (AD8232 or similar with electrodes)
- MAX30100 Heart Rate and SpO2 Sensor Module
- NTC Thermistor (10kΩ) with 10kΩ fixed resistor
- MPU6050 Accelerometer and Gyroscope Module
- 2-Cell Li-ion Battery (7.4V, 2000-5000mAh)
- Buck Converter Module (to 5V/3.3V)
- Jumper Wires and Connectors
- Wearable Enclosure (chest strap or armband)
- ECG Electrodes (disposable or reusable)
- Power Switch

Software Requirements

- Arduino IDE with ESP32 Board Package
- BluetoothSerial Library (built-in)
- Wire Library for I2C (built-in)
- MAX30100 Library (e.g., MAX30100lib)
- MPU6050 Library (e.g., Adafruit_MPU6050)
- Math Library for calculations
- Google Colab for Logistic Regression Training
- Python with scikit-learn, numpy, pandas
- Serial Bluetooth Terminal App (Android)

Applications

- Home Health Monitoring: Continuous monitoring of cardiac and respiratory health
- Elderly Care: Fall detection and vital signs monitoring for seniors
- Telemedicine: Remote patient monitoring during virtual consultations
- Post-Surgery Recovery: Monitoring patients after cardiac procedures
- Sports and Fitness: Athlete health and performance monitoring
- Chronic Disease Management: Long-term monitoring of cardiac conditions
- Research: Data collection for health studies
- Rural Healthcare: Portable monitoring in areas with limited clinical access

Performance Metrics

- ECG Sampling Rate: 250 Hz
- Heart Rate Range: 30-250 BPM
- RR Interval Accuracy: ±5 ms
- SpO2 Accuracy: ±2% (calibrated)
- Temperature Accuracy: ±0.5°C
- Fall Detection Response: < 1 second
- Bluetooth Range: Up to 10 meters
- ML Inference Time: < 10 ms
- Data Update Rate: 1 Hz

Conclusion

The ESP32-based healthcare monitoring system successfully demonstrates an integrated solution for continuous monitoring of critical health parameters using affordable, portable, and intelligent embedded technology. The system combines ECG signal processing for RR interval extraction and cardiac classification (Normal, Tachycardia, Bradycardia) using logistic regression edge AI, SpO2 monitoring via MAX30100, body temperature measurement using NTC thermistor, and fall detection using MPU6050. All data is transmitted in real-time to a mobile app via the ESP32's built-in Bluetooth, enabling continuous monitoring by the user, family, or caregivers. The 2-cell Li-ion battery with buck converter ensures portable operation with stable 5V power. The edge AI approach eliminates cloud dependency, ensuring privacy and low-latency response. This project addresses critical gaps in affordable healthcare monitoring by providing a comprehensive, customizable, and open-source solution suitable for home use, elderly care, and telemedicine. Future enhancements could include GSM-based emergency SMS alerts, SD card data logging, cloud sync via WiFi, additional sensors (blood pressure, glucose), deep learning models for improved accuracy, and integration with electronic health records. This project establishes a foundation for intelligent, accessible, and life-saving healthcare technology for diverse populations.

Battery



This LG INR18650 M26 2600mAh Lithium-Ion Battery gives value for your money. It comes with a rated voltage of 3.7 volts and a capacity of 2600mAh. It is a single cell, compact, and powerful battery cell with 2600 mAh capacity. It is very convenient to install in your project to fulfill the 3.7 Volt requirement with high capacity.

The battery terminals can use in any compatible battery adapter/holder or it can be permanently soldered to your applications power source wires.

Features :

High energy density

High working voltage for single battery cells.

Pollution-free

Long cycle life

self-discharge.

Lightweight, small size

Shape: Cylindrical Battery

Battery Type: Lithium-Ion Battery

High performance and capacity

Flat top to suit many devices fitting.

MAX30100 HEART RATE SENSOR



Heart Rate click carries Maxim's MAX30100 integrated pulse oximetry and a heart-rate sensor. It's an optical sensor that derives its readings from emitting two wavelengths of light from two LEDs – a red and an infrared one – then measuring the absorbance of pulsing blood through a photodetector. This particular LED colour combination is optimized for reading the data through the tip of one's finger.

The signal is processed by a low-noise analog signal processing unit and communicated to the target MCU through the mikroBUS I2C interface. Developers of end-user applications should note that the readings can be negatively impacted by excess motion and changes in temperature. Also, too much pressure can constrict capillary blood flow and therefore diminish the reliability of the data. A programmable INT pin is also available. The operates at the 3.3V power supply.

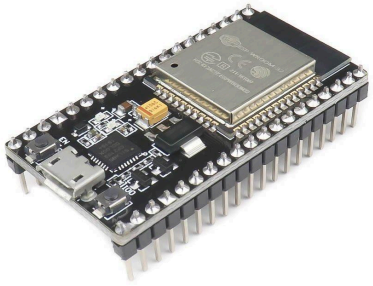
Main Applications:

Medical Monitoring Devices

Fitness Assistant Devices

Wearable Devices

ESP32



About

1. Ultra-low-power management.
2. 4 MB Flash.
3. Current: 80 mA.
4. Supply Voltage: 2.2 V ~ 3.6 V.
5. Data Rate: 54 Mbps.
6. Frequency: 2.4 GHz.

Description

This is ESP WROOM 32 MCU Module. ESP WROOM 32 is a powerful, generic WiFi-BT-BLE MCU module that targets a wide variety of applications, ranging from low-power sensor networks to the most demanding tasks, such as voice encoding, music streaming, and MP3 decoding.

At the core of this module is the ESP32S chip, which is designed to be scalable and adaptive. There are 2 CPU cores that can be individually controlled or powered, and the clock frequency is adjustable from 80 MHz to 240 MHz.

The user may also power off the CPU and make use of the low-power coprocessor to constantly monitor the peripherals for changes or crossing of thresholds. ESP32S integrates a rich set of peripherals, ranging from capacitive touch sensors, Hall sensors, low-noise sense amplifiers, SD card interface, Ethernet, high-speed SDIO/SPI, UART, and I^2C.

Using Bluetooth, users can connect to their phone or broadcast low energy beacons for its detection. The use of Wi-Fi enables a large physical range, as well as a direct connection to

the internet via a Wi-Fi router. Perfect for wearable electronic or battery-powered applications, the ESP32 chip uses less than 5μA.
In addition, this module can support data rates of up to 150 Mbps and 22 dBm output power at the PA in order to allow for the widest physical range.

ESP32

1. ESP32 is a single chip 2.4 GHz Wi-Fi and Bluetooth combo chip designed with TSMC ultra-low power 40 nm technology. It is designed and optimized for the best power performance, RF performance, robustness, versatility, features, and reliability, for a wide variety of applications, and different power profiles.
2. The ESP32 is the most integrated solution for Wi-Fi + Bluetooth applications in the industry with less than 10 external components. ESP32 integrates the antenna switch, RF balun, power amplifier, low noise receive amplifier, filters, and power management modules. As such, the entire solution occupies minimal Printed Circuit Board (PCB) area.
3. ESP32 is designed for mobile, wearable electronics, and Internet of Things (IoT) applications. It has many features of the state-of-the-art low power chips, including fine resolution clock gating, power modes, and dynamic power scaling.

Application :

1. Universal low power IoT sensor hub.
2. Home automation.
3. Universal low power IoT recorder.
4. Mesh network.
5. Video streaming of the camera.
6. Industrial wireless control.
7. OTT TV box / set-top box device.
8. Baby monitor.
9. Smart Socket.
10. Sensor networks.
11. Wi-Fi toys: Counters, toys, Anti-lost device.
12. Wearable electronic products.
13. Wi-Fi speech recognition device.
14. Wi-Fi location-aware devices.

Features :

1. Integrated 520 KB SRAM.
2. Hybrid Wi-FI & Bluetooth.
3. High level of integration.
4. Ultra-low-power management.
5. 4 MB Flash.
6. On-board PCB antenna.

AD8232 ECG Monitor Sensor Module



ECG Monitor Sensor Module is based on AD8232 Analog Device IC. This is a cost-effective ECG Sensor used to measure the electrical activity of the heart. This electrical activity can be charted as an ECG or Electrocardiogram and output as an analog reading. ECGs can be extremely noisy, out AD8232 Single Lead Heart Rate Monitor acts as an op amp to help obtain a clear signal. This sensor can be connected to an Arduino/Raspberry Pi, etc. Sample codes are easily available on the internet.

The AD8232 is an integrated signal conditioning block for ECG and other biopotential measurement applications. It is designed to extract, amplify, and filter small biopotential signals in the presence of noisy conditions, such as those created by motion or remote electrode placement.

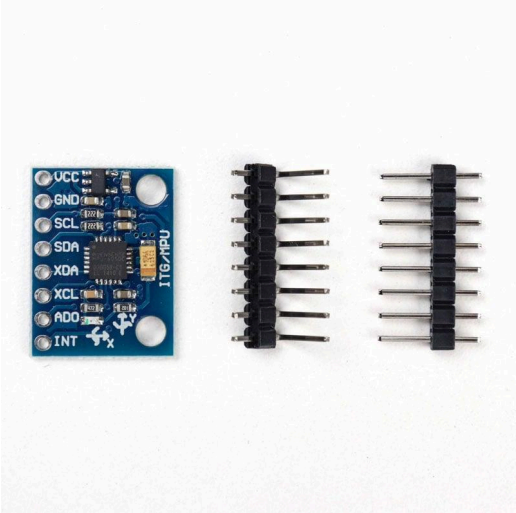
The AD8232 module breaks out nine connections from the IC that you can solder pins, wires, or other connectors to. SDN, LO+, LO-, OUTPUT, 3.3V, GND provide essential pins for operating this monitor with an Arduino or other development board. Also provided on this board are RA (Right Arm), LA (Left Arm), and RL (Right Leg) pins to attach and use your own custom sensors. Additionally, there is an LED indicator light that will pulsate to the rhythm of a heart beat.

Note: This product is NOT a industry grade medical device and is not intended to be used as such or as an accessory to such nor diagnose or treat any conditions.

Features of AD8232 ECG Monitor Sensor Module:-

- Operating Voltage - 3.3V
- Analog Output
- Leads-Off Detection
- Shutdown Pin
- LED Indicator
- 3.5mm Jack for Biomedical Pad Connection or Use 3 pin header

MPU-6050 3-Axis Accelerometer and Gyro Sensor



The **MPU-6050** 3-Axis Accelerometer and Gyro Sensor module use **MPU-6050** which is a little piece of motion processing tech. The MPU6050 datasheet is available in the attachment section to get this visit.

There are different types of magnetometer are available the basic differences between **MPU6000**, **MPU6050** and **MPU6500** is:

MPU6000	**MPU6050**	**MPU6500**
1. A sampling rate of 8 kHz.	1. A sampling rate of 8 kHz.	1. A sampling rate of 32 kHz.
2. Supports SPI and I^2C interface. But I^2C is too slow to handle 8 kHz gyro update.	2. Supports I^2C interface communication protocol.	2. Supports both I^2C and SPI interface.
3. Vibration Sensitivity is better than MPU6500.	3. Vibration Sensitivity is better than MPU6500 but the speed of operation is less than MPU6000.	3. It is more susceptible to vibrations, so the need for some vibration isolation methods. It is faster than both MPU.

The **MPU6050** devices combine a 3-axis gyroscope and a 3-axis accelerometer on the same silicon together with an onboard Digital Motion Processor (DMP) capable of processing complex 9-axis MotionFusion algorithms. To get more insights about specifications visit the MPU6050 datasheet.

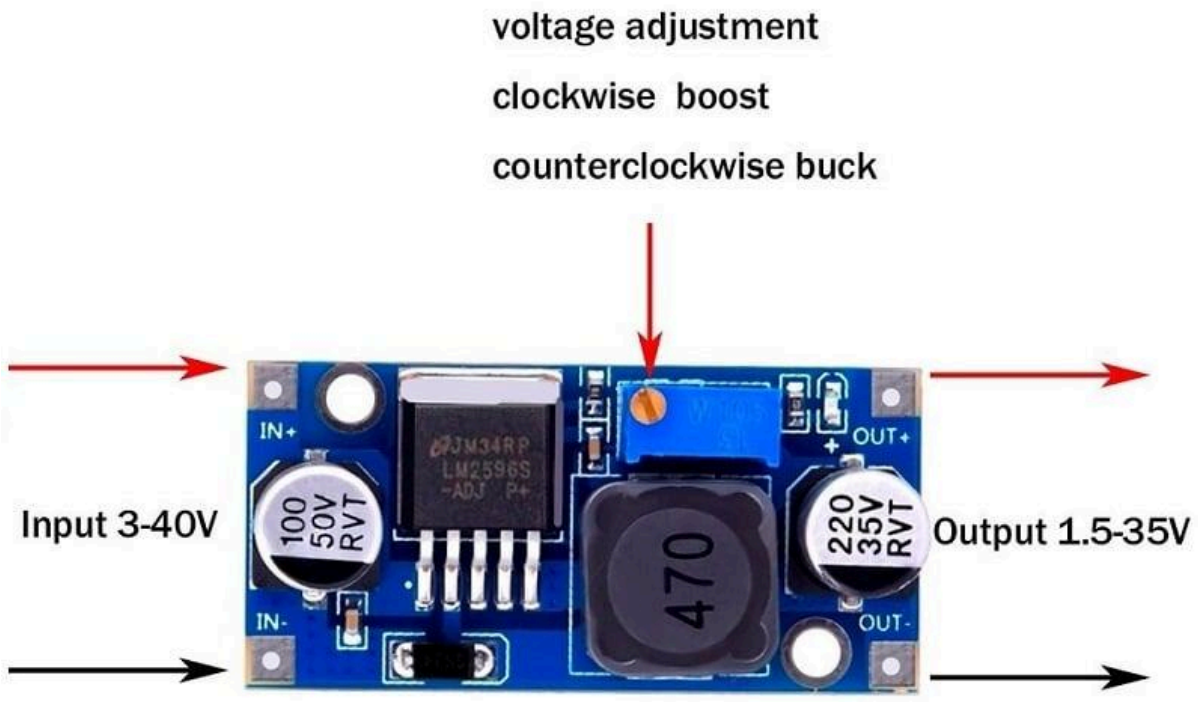
Features :

1. Chip built-in 16bit AD converter, 16-bit data output.
2. I2C Digital-output of 6 or 9-axis MotionFusion data in rotation matrix, quaternion, Euler Angle, or raw data format.
3. Selectable Solder Jumpers on CLK, FSYNC, and AD0.
4. Digital Motion Processing™ (DMP™) engine offloads complex MotionFusion, sensor timing synchronization, and gesture detection.
5. Embedded algorithms for run-time bias and compass calibration. No user intervention is required.
6. Digital-output temperature sensor.

LM2596S with SMD LED DC-DC Step-Down Power Supply

DC-DC Buck Converter Step Down Module LM2596 Power Supply is a step-down(buck) switching regulator, capable of driving a 3-A load with excellent line and load regulation. These devices are available in fixed output voltages of 3.3 V, 5 V, 12 V, and an adjustable output version.

The LM2596 series operates at a switching frequency of 150kHz, thus allowing smaller sized filter components than what would be required with lower frequency switching regulators.



This is an LM2596 DC-DC buck converter step-down power module with the high-precision potentiometer, capable of driving a load up to 3A with high efficiency, which can work with Freeduino UNO, other mainboards, and basic modules. When the output current keeps greater than 2.5A (or output power greater than 10W), please add a heat sink to it.

This device is internally compensated to minimize the number of external components to simplify the power supply design.

Since LM2596 converter is a switch-mode power supply, its efficiency is significantly higher in comparison with popular three-terminal linear regulators, especially with higher input voltages. The LM2596 operates at a switching frequency of 150 kHz thus allowing smaller sized filter components than what would be needed with lower frequency switching regulators.

Features:

1. Input voltage:3-40V
2. Output voltage:1.5-35V(Adjustable)
3. Output current: Rated current is 2A, maximum 3A(Additional heatsink is required)
4. Module Properties: non-isolated constant voltage module
5. Rectification: non-synchronous rectification
6. Short circuit protection: current limiting, since the recovery

---

SOFTWARE DISCRIPTION

Arduino ide is the software used to write-compile-upload program to esp32.Its a open source software

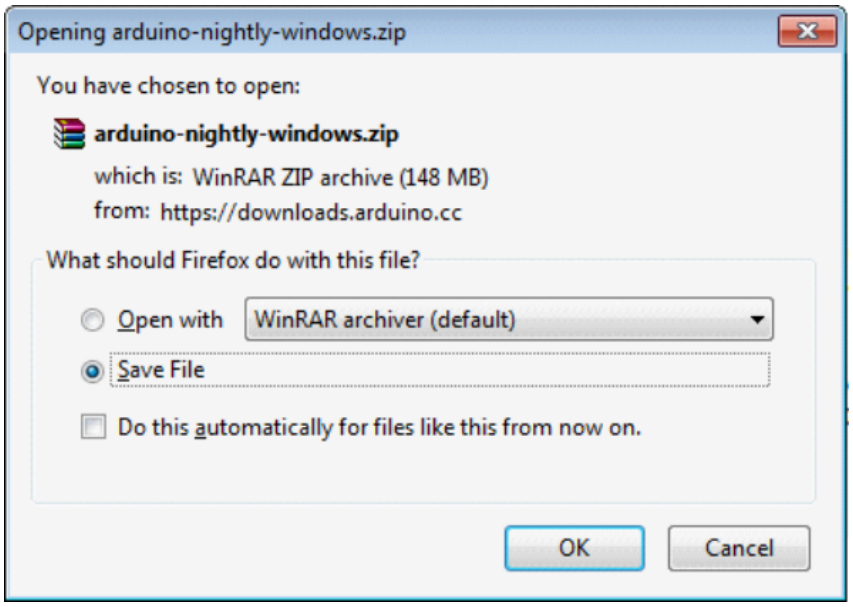
- **Procedure to Install Arduino Software (IDE)**

Step 1 — First we must have an Arduino board and a USB cable. In case we use Arduino UNO, Arduino Duemilanove, Nano, Arduino Mega 2560, or Diecimila, we will need a standard USB cable (A plug to B plug), the kind we would connect to a USB printer as shown in Fig. 3.1.5.1 (a). In case we use Arduino Nano, we will need an A to Mini-B cable instead as shown in Fig. 3.1.5.1 (b).



Fig 3.1.5.1 (b) A to Mini-B Cable

Step 2 — Download Arduino IDE Software.



One can get different versions of Arduino IDE from the Download page on the Arduino Official website. We must select our software, which is compatible with our operating system (Windows, IOS, or Linux). Unzip the file, after downloading it completely.

Fig. 3.1.5.2 Downloading Arduino IDE

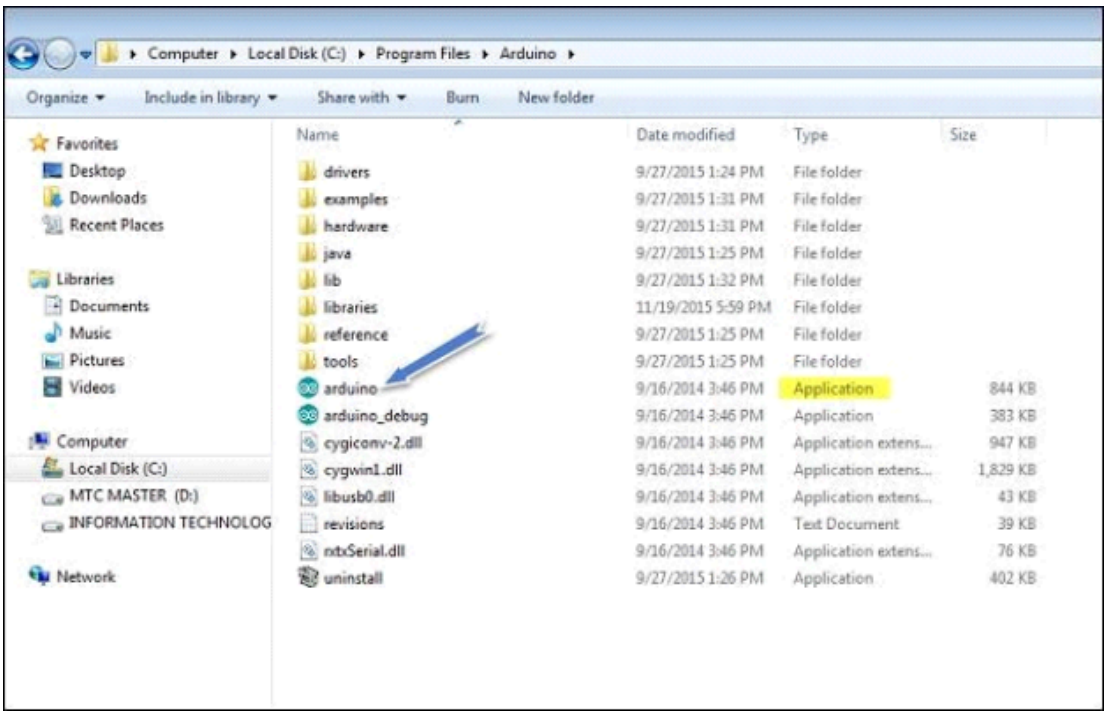
Step 3 — Power up your board.

The Arduino Uno, Mega, Duemilanove and Arduino Nano automatically draw power from either, the USB connection to the computer or an external power supply. If you are using an Arduino Diecimila, you have to make sure that the board is configured to draw power from the USB connection. The power source is selected with a jumper, a small piece of

plastic that fits onto two of the three pins between the USB and power jacks. Check that it is on the two pins closest to the USB port.

Connect theesp32 board to computer using the USB cable. The green power LED (labeled PWR) should glow.

Step 4 — Launch Arduino IDE.



After your Arduino IDE software is downloaded, you need to unzip the folder. Inside the folder, you can find the application icon with an infinity label (application.exe). Double-click the icon to start the IDE.

Fig. 3.1.5.4 Launching Arduino IDE

Step 5 — Open your first project.

Once the software starts, we have two options:

- Create a new project.

  To create a new project, select File →**New,** as shown in Fig. 3.1.5.5 (a).

- Open an existing project example.

  To open an existing project example, select File → Example → Basics → Blink, as shown in Fig. 3.1.5.5 (b).

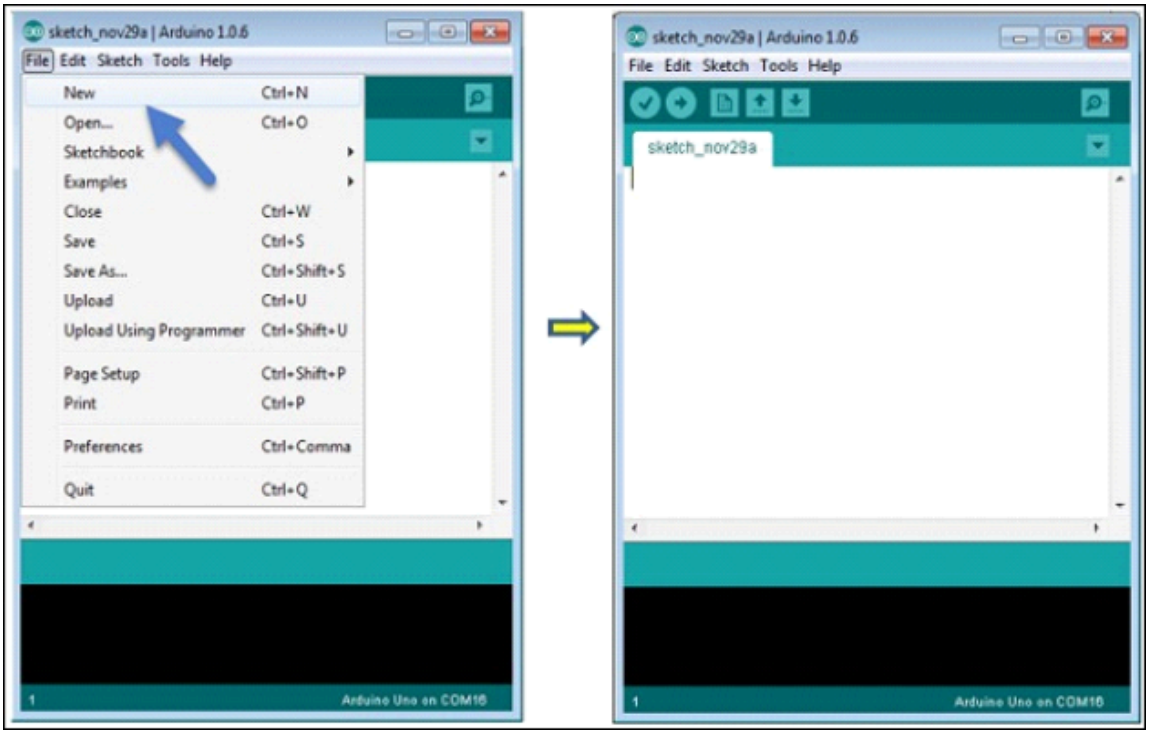


Fig. 3.1.5.5 (a) Creating a New Project

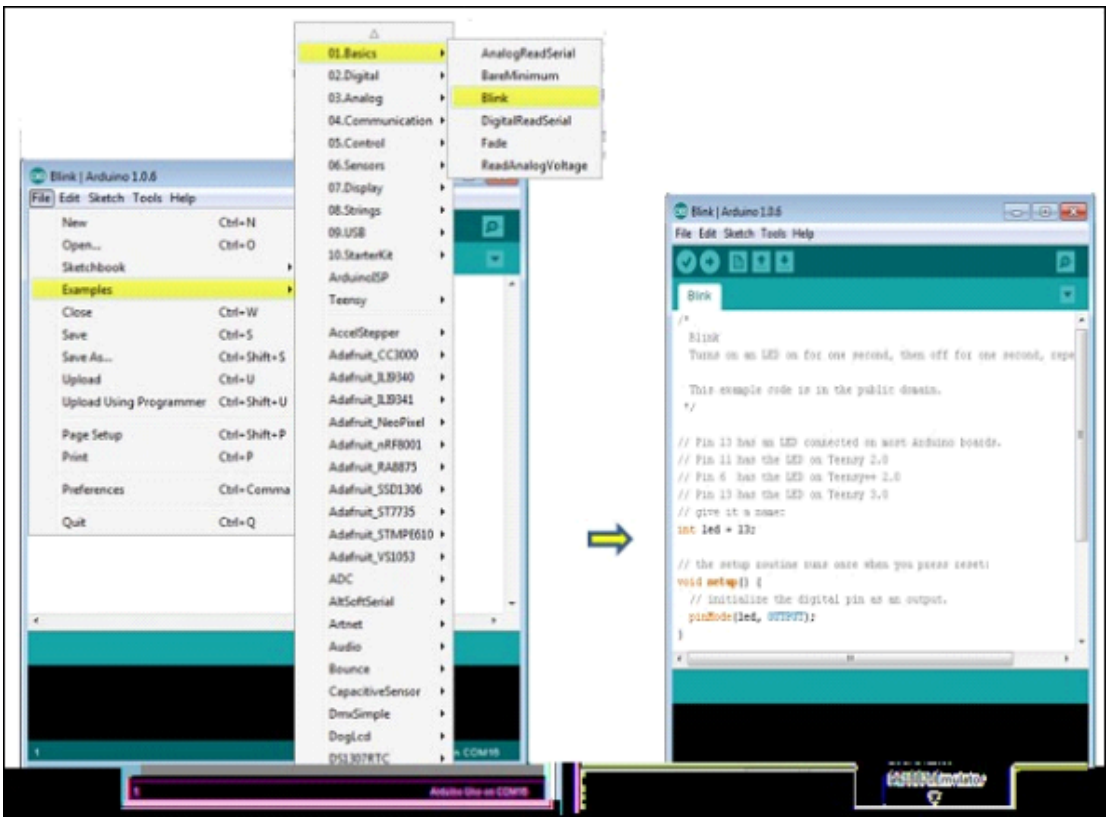


Fig. 3.1.5.5 (b) Opening an Existing Project

Here, we are selecting just one of the examples with the name **Blink**. It turns the LED on and off with some time delay. We can select any example from the list.

Step 6 — Select the respectiveesp32 board.

To avoid any error while uploading our program to the board, we must select the correctesp32 board name, which matches with the board connected to our computer.

Go to Tools → Board and select the board.

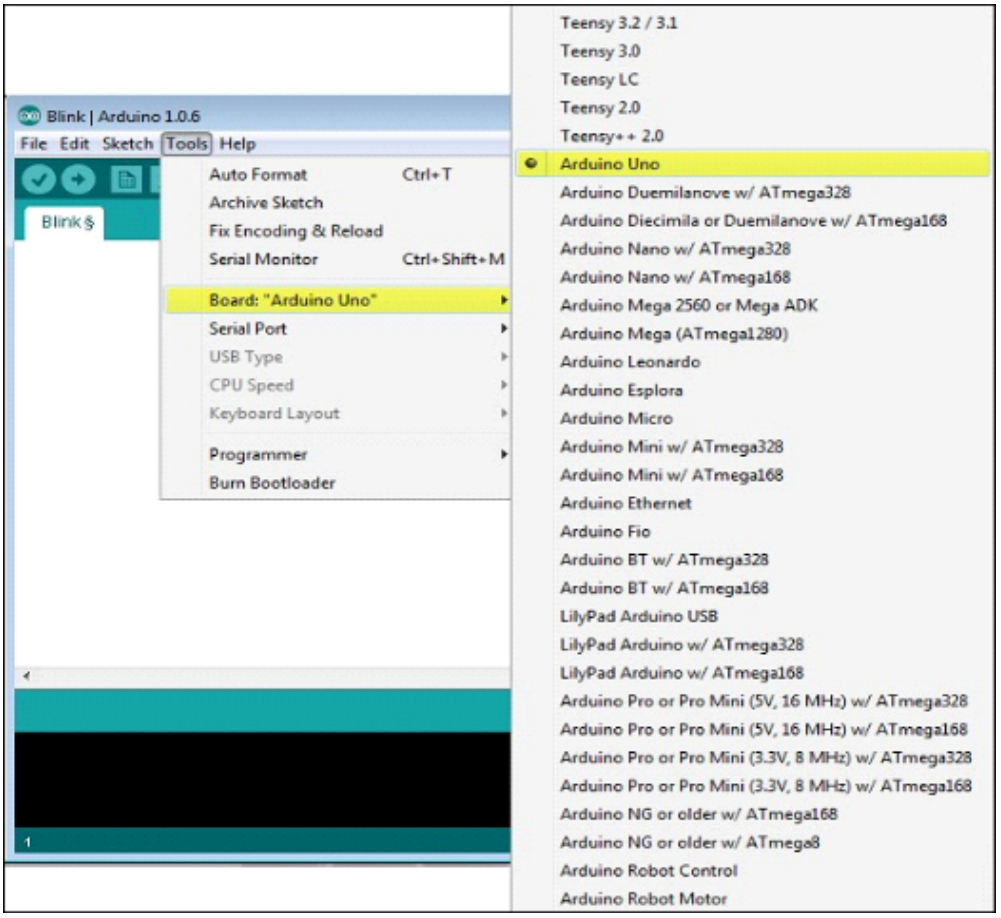


Fig. 3.1.5.6 Selecting theesp32 Board

Step 7 — Select the serial port.

Select the serial device of theesp32 board. Go to **Tools → Serial Port** menu.

This is likely to be COM3 or higher (COM1 and COM2 are usually reserved for hardware serial ports). To find out, you can disconnect youresp32 board and re-open the menu, the entry that disappears should be of theesp32 board. Reconnect the board and select that serial port.

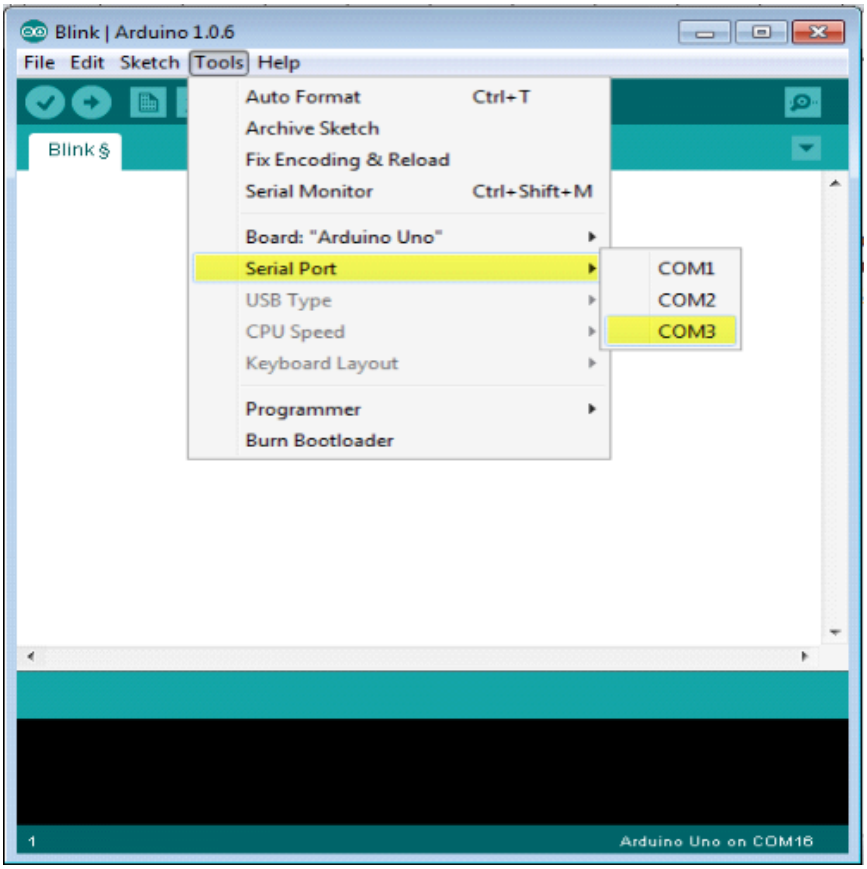
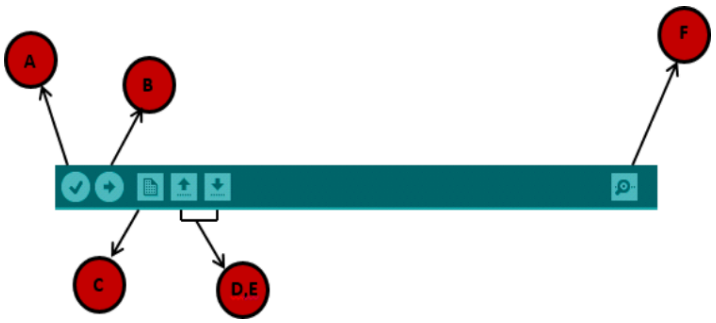


Fig. 3.1.4.7 Selecting the Serial Port

Step 8 — Upload the program to the board.



Before explaining how to upload our program to the board, we should know the function of each symbol appearing in the Arduino IDE toolbar.

Fig. 3.1.4.8 Arduino IDE Toolbar

A – Used to check if there is any compilation error. **B** – Used to upload a program to theesp32 board. **C** – Shortcut used to create a new sketch.

D – Used to directly open one of the example sketch.

E – Used to save your sketch.

F – Serial monitor used to send and receive the serial data from the board.

Now, simply click the "Upload" button in the environment. Wait few seconds; we will see the RX and TX LEDs on the board, flashing. If the upload is successful, the message "Done uploading" will appear in the status bar.

- **Program Structure**

In this section, we will study in depth, the esp32 program structure and we will learn more new terminologies used in the esp32 world. The Arduino software is open-source. The source code for the Java environment is released under the GPL and the C/C++ microcontroller libraries are under the LGPL. Arduino programs are called sketch.

Arduino programs can be divided in three main parts: **Structure, Values** (variables and constants), and **Functions**. Let us learn about the Arduino software program, step by step, and how to write the program without any syntax or compilation error.

Let us start with the **Structure**. Software structure consist of two main functions:

- Setup( ) function
- Loop( ) function

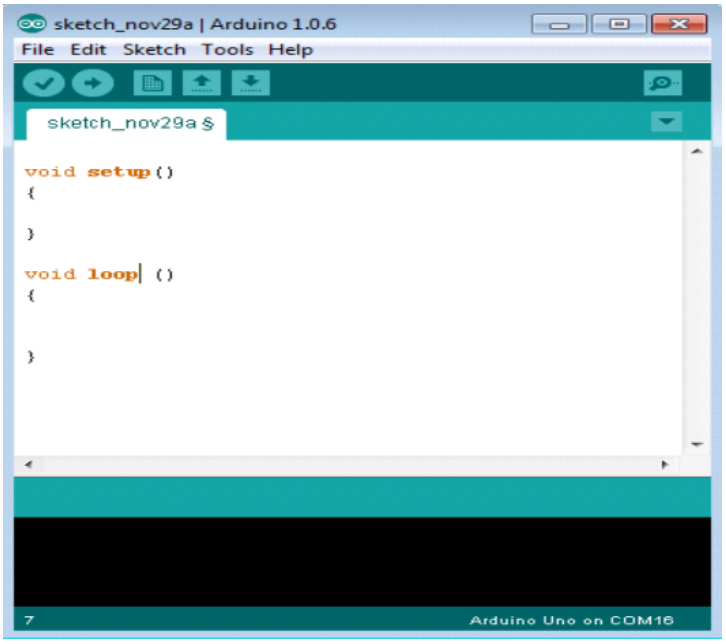


Fig. 3.1.6 Structure of a Sketch

The **setup ()** function is called when a sketch starts. It is used to initialize the variables, pin modes, start using libraries, etc. The setup function will only run once, after each power up or reset of the esp32 board.

After creating a **setup ()** function, which initializes and sets the initial values, the **loop ()** function does precisely what its name suggests, and loops consecutively, allowing our program to change and respond. It actively controls the esp32 board.

- **Programming using Embedded C**

C is a high-level programming language intended for system programming. Embedded C is an extension that provides support for developing efficient programs for embedded devices. Yet, it is not a part of the C language. In our Internship program, we employed Embedded C programs to write sketches to be dumped on esp32 Uno.

- **Introduction to Embedded C**

Embedded C programming language is an extension to the traditional C programming language that is used in embedded systems. The embedded C programming language uses the same syntax and semantics as the C programming language.

The only extension in the Embedded C language from normal C Programming Language is the I/O Hardware Addressing, fixed-point arithmetic operations, accessing address spaces, etc.

- **Basic Structure of Embedded C Program:**

The embedded C program has a structure similar to C programming. The five layers of Embedded C programming structure are:

- Comments
- Pre-processor directives
- Global declaration
- Local declaration
- Main function()

The whole code follows the below outline. This is the basic structure of the embedded c program. Each code has a similar outline. Now let us learn about each of this layer in detail.

Outline of an Embedded C code is as shown below:

- Multiline Comments Denoted using /*……*/
- Single Line Comments Denoted using //
- Pre-processor Directives #include<…> or #define
- Global Variables Accessible anywhere in the program
- Function Declarations Declaring Function
- Main Function Main Function, execution begins here

7. {

- Local Variables Variables confined to main function
- Function Calls Calling other Functions
- Infinite Loop Like while (1) or for (;;)
- Statements . . . . .

12. ….

13. ….

14. }

15. Function Definitions Defining the Functions

16. {

- Local Variables Local Variables confined to this Function
- Statements . . . . .

19. ….

20. ….

21. }

- **Comment Section:** Comments are simple readable text, written in code to make it more understandable to the reader. Usually comments are written in // or /* */.

  Example: //Test program

- **Pre-processor Directives Section:** The Pre-Processor directives tell the compiler which files to look in to find the symbols that are not present in the program.

For Example, in 8051 Keil compiler we use,

- #include<reg51.h>

- **Global Declaration Section:** The global variables and the user-defined functions are declared in this part of the code. They can be accessed from anywhere.

  1. void delay (int);

- **Local Declaration Section:** These variables are declared in the respective functions and cannot be used outside the main function.

- **Main Function Section:** Every C programs need to have the main function. So does an embedded C program. Each main function contains 2 parts. A declaration part and an Execution part. The declaration part is the part where all the variables are declared. The execution part begins with the curly brackets and ends with the curly close bracket. Both the declaration and execution part are inside the curly braces.

  Example:

  1. void main(void) // Main Function 2. {
  3. P1 = 0x00;
  4. while(1)
  5. {

```
6.      P1 = 0xFF;
7.      delay(1000);
8.      P1 = 0x00;
9.      delay(1000);
10.     }
11.     }
```

- **Function Definition Section:** The function is defined in this section.

- **Arduino Code libraries**
- **Library Structure**

- A library is a folder comprised with C++ (.cpp) code files and C++ (.h) header files.
- The .h file describes the structure of the library and declares all its variables and functions.
- The .cpp file holds the function implementation.

- **Importing Libraries**

The first thing to do is to find the library we want to use out of the many libraries available online. After downloading it to our computer, we just need to open Arduino IDE and click on Sketch > Include Library > Manage Libraries. We can then select the library we want to import. Once the process is complete the library will be available in the sketch menu.

- **Arduino Code Explanation**

Arduino code is written in C++ with an addition of special methods and functions. C++ is a human-readable programming language. When we create a sketch (the name given to esp32 code files), it is processed and compiled to machine language.

- **Code Structure**

The basic concepts which one should know to write a program on Arduino IDE are discussed below:

- **Libraries**

In Arduino, much like other leading programming platforms, there are built-in libraries that provide basic functionality. In addition, its possible to import other libraries and expand the Arduino board capabilities and features. These libraries are roughly divided into libraries that interact with a specific component or those that implement new functions.

To import a new library, we need to go to Sketch > Import Library

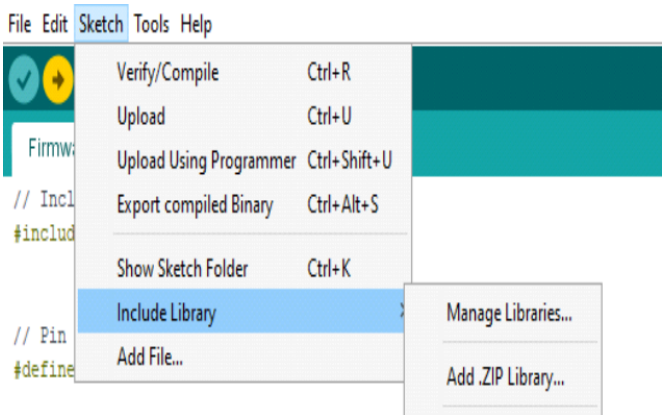


Fig. 3.2.2.1 Including Libraries on IDE

In addition, at the top of our file, we have to use #include to include external libraries. We can also create custom libraries to use in isolated sketches.

- **Pin Definitions**

To use the esp32 pins, we need to define which pin is being used and its functionality. A convenient way to define the used pins is by using:

#define pinName pinNumber.

The functionality is either input or output and is defined using the pinMode () method in the setup section.

- **Declarations**

- **Variables**

Whenever were using esp32, we need to declare global variables and instances to be used later on. In a nutshell, a variable allows us to name and store a value to be used in the

future. For example, we would store data acquired from a sensor in order to use it later. To declare a variable we simply define its type, name and initial value. Its worth mentioning that declaring global variables isnt an absolute necessity. However, its advisable to declare variables to make it easy to utilize our values further down the line.

- **Instances**

In software programming, a class is a collection of functions and variables that are kept together in one place. Each class has a special function known as a constructor, which is used to create an instance of the class. In order to use the functions of the class, we need to declare an instance for it.

- **Setup()**

Every esp32 sketch must have a setup function. This function defines the initial state of the esp32 upon boot and runs only once.

Here well define the following:

- Pin functionality using the pinMode function
- Initial state of pins
- Initialize classes
- Initialize variables
- Code logic

- **Loop()**

The loop function is also a must for every esp32 sketch and executes once setup () is complete. It is the main function and as its name hints, it runs in a loop over and over again. The loop describes the main logic of our circuit.

- **Code Logic**

The basic esp32 code logic is an if-then structure and can be divided into 4 blocks:

- **Setup** - will usually be written in the setup section of the esp32 code, and performs things that need to be done only once, such as sensor calibration.

- **Input** - at the beginning of the loop, read the inputs. These values will be used as conditions (if) such as the ambient light reading from an LDR using analogRead ().
- **Manipulate Data** - this section is used to transform the data into a more convenient form or perform calculations. For instance, the AnalogRead () gives a reading of 0-1023 which can be mapped to a range of 0-255 to be used for PWM. (See analogWrite ())
- **Output** - this section defines the final outcome of the logic (then) according to the data calculated in the previous step. Looking at an example of the LDR and PWM, turns on an LED only when the ambient light level goes below a certain threshold.

- **From Software to Hardware**

There is a lot to be said of esp32s software capabilities, but its important to remember that the platform is comprised of both software and hardware. The two work in tandem to run a complex operating system.

Code → Compile → Upload → Run

At the core of esp32, is the ability to compile and run the code.

After writing the code in the IDE we need to upload it to the esp32. Clicking the Upload button (the right-facing arrow icon), will compile the code and upload it if it passed compilation. Once the upload is complete, the program will start running automatically.